

A* Search Algorithm

Zarif Ishmam
2305035

Tonmoy Biswas
2305038

Proteek Rosul
2305042

CSE200 · CSE, BUET



1

Introduction to Pathfinding and A*

Zarif Ishmam · 2305035

2

Complexity Analysis and Heuristics

Proteek Rosul · 2305042

3

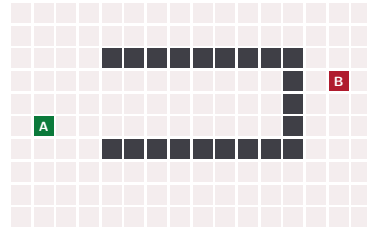
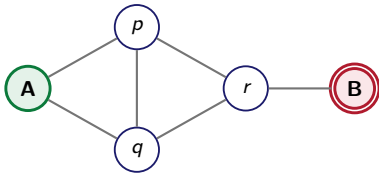
Worked Simulation

Tonmoy Biswas · 2305038



Pathfinding

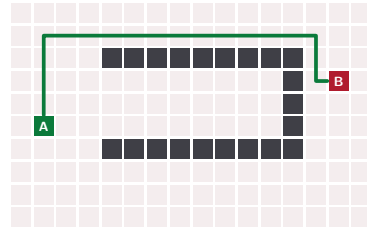
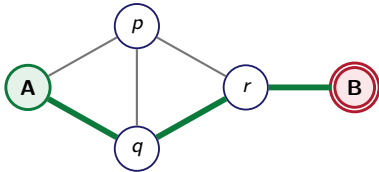
The problem of simply finding a **valid path** from any point **A** to any point **B** in a graph.





Pathfinding

The problem of simply finding a **valid path** from any point **A** to any point **B** in a graph.



Where pathfinding is used



- Road navigation
- Game AI and NPC movement
- Networks and packet routing
- Protein folding
- ...and many more
Puzzles, Robotics, Dependency resolution

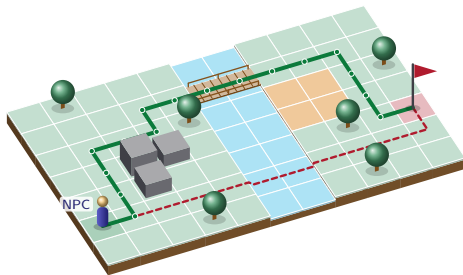
¹ Image source: <https://www.nature.com/articles/s41467-026-75254-8>

Where pathfinding is used



- Road navigation
- Game AI and NPC movement
- Networks and packet routing
- Protein folding
- ...and many more
Puzzles, Robotics, Dependency resolution

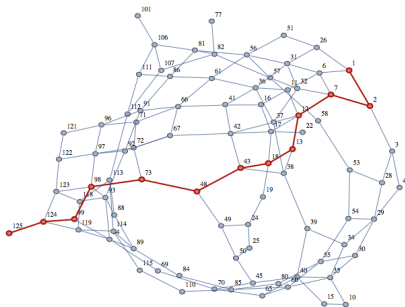
¹ Image source: <https://www.nature.com/articles/s41467-026-75254-8>



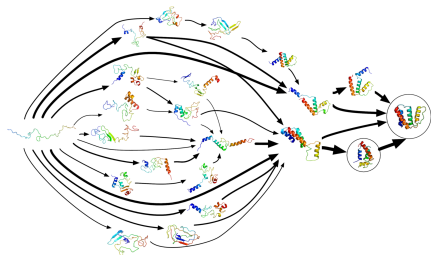
step cost: grass 1 mud 3 water 5 rock, tree: blocked
— cheapest: 17 steps, cost 17 - - - shortest: 11 steps, cost 19

- Road navigation
- **Game AI and NPC movement**
- Networks and packet routing
- Protein folding
- ...and many more
Puzzles, Robotics, Dependency resolution

Where pathfinding is used

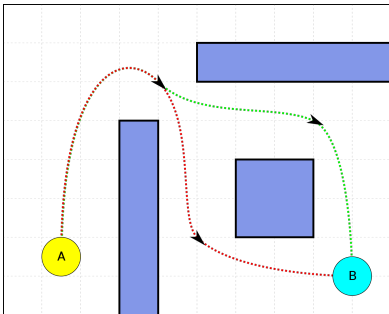


- Road navigation
 - Game AI and NPC movement
 - **Networks and packet routing**
 - Protein folding
 - ...and many more
- Puzzles, Robotics, Dependency resolution



- Road navigation
- Game AI and NPC movement
- Networks and packet routing
- **Protein folding**
- ...and many more
Puzzles, Robotics, Dependency resolution

¹ Image source: https://en.wikipedia.org/wiki/Protein_folding#/media/File:ACBP_MSM_from_Folding@home.tiff



- Road navigation
- Game AI and NPC movement
- Networks and packet routing
- Protein folding
- **...and many more**
Puzzles, Robotics, Dependency resolution

Two requirements



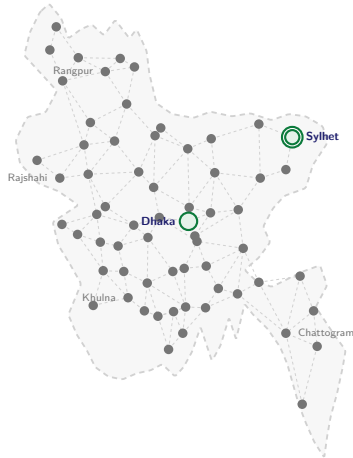
1

Shortest path

2

Efficient algorithm

A real graph — the 64 districts of Bangladesh



Let's look at an example of a path from **Dhaka** to **Sylhet**.

Shortest Path — Dijkstra's Algorithm



```
1 Dijkstra( $G, s, t$ )
2   for each  $v \in V$ :  $g[v] \leftarrow \infty$ ,  $\pi[v] \leftarrow \text{nil}$ 
3    $g[s] \leftarrow 0$ 
4    $Q \leftarrow$  priority queue keyed by  $g$ , holding  $\{s\}$ 
5    $S \leftarrow \emptyset$ 
6   while  $Q \neq \emptyset$ 
7      $u \leftarrow \text{Extract-Min}(Q)$ 
8     if  $u = t$ :
9       return Reconstruct( $\pi, t$ )
10     $S \leftarrow S \cup \{u\}$ 
11    for each  $v$  adjacent to  $u$ 
12      if  $v \in S$ : continue
13      if  $g[u] + w(u, v) < g[v]$ 
14         $g[v] \leftarrow g[u] + w(u, v)$ ,  $\pi[v] \leftarrow u$ 
15        Insert-or-Decrease-Key ( $Q, v, g[v]$ )
```



```
1 Dijkstra( $G, s, t$ )
2   for each  $v \in V$ :  $g[v] \leftarrow \infty$ ,  $\pi[v] \leftarrow \text{nil}$ 
3    $g[s] \leftarrow 0$ 
4    $Q \leftarrow$  priority queue keyed by  $g$ , holding  $\{s\}$ 
5    $S \leftarrow \emptyset$ 
6   while  $Q \neq \emptyset$ 
7      $u \leftarrow \text{Extract-Min}(Q)$ 
8     if  $u = t$ :
9       return Reconstruct( $\pi, t$ )
10     $S \leftarrow S \cup \{u\}$ 
11    for each  $v$  adjacent to  $u$ 
12      if  $v \in S$ : continue
13      if  $g[u] + w(u, v) < g[v]$ 
14         $g[v] \leftarrow g[u] + w(u, v)$ ,  $\pi[v] \leftarrow u$ 
15        Insert-or-Decrease-Key ( $Q, v, g[v]$ )
```

How Dijkstra picks



```
1 Dijkstra( $G, s, t$ )
2   for each  $v \in V$ :  $g[v] \leftarrow \infty$ ,  $\pi[v] \leftarrow \text{nil}$ 
3    $g[s] \leftarrow 0$ 
4    $Q \leftarrow$  priority queue keyed by  $g$ , holding  $\{s\}$ 
5    $S \leftarrow \emptyset$ 
6   while  $Q \neq \emptyset$ 
7      $u \leftarrow \text{Extract-Min}(Q)$ 
8     if  $u = t$ :
9       return Reconstruct( $\pi, t$ )
10     $S \leftarrow S \cup \{u\}$ 
11    for each  $v$  adjacent to  $u$ 
12      if  $v \in S$ : continue
13      if  $g[u] + w(u, v) < g[v]$ 
14         $g[v] \leftarrow g[u] + w(u, v)$ ,  $\pi[v] \leftarrow u$ 
15        Insert-or-Decrease-Key ( $Q, v, g[v]$ )
```

How Dijkstra picks

- Start from the source



```
1 Dijkstra( $G, s, t$ )
2   for each  $v \in V$ :  $g[v] \leftarrow \infty$ ,  $\pi[v] \leftarrow \text{nil}$ 
3    $g[s] \leftarrow 0$ 
4    $Q \leftarrow$  priority queue keyed by  $g$ , holding  $\{s\}$ 
5    $S \leftarrow \emptyset$ 
6   while  $Q \neq \emptyset$ 
7      $u \leftarrow \text{Extract-Min}(Q)$ 
8     if  $u = t$ :
9       return Reconstruct( $\pi, t$ )
10     $S \leftarrow S \cup \{u\}$ 
11    for each  $v$  adjacent to  $u$ 
12      if  $v \in S$ : continue
13      if  $g[u] + w(u, v) < g[v]$ 
14         $g[v] \leftarrow g[u] + w(u, v)$ ,  $\pi[v] \leftarrow u$ 
15        Insert-or-Decrease-Key ( $Q, v, g[v]$ )
```

How Dijkstra picks

- Start from the source
- In each iteration, Picks the **locally best** node so far

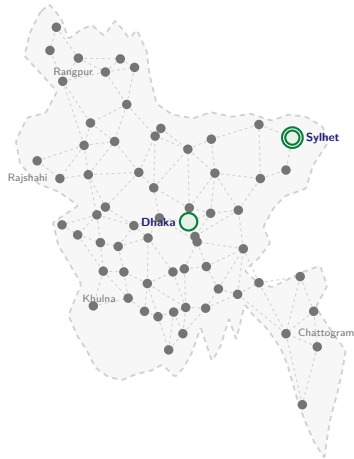


```
1 Dijkstra( $G, s, t$ )
2   for each  $v \in V$ :  $g[v] \leftarrow \infty$ ,  $\pi[v] \leftarrow \text{nil}$ 
3    $g[s] \leftarrow 0$ 
4    $Q \leftarrow$  priority queue keyed by  $g$ , holding  $\{s\}$ 
5    $S \leftarrow \emptyset$ 
6   while  $Q \neq \emptyset$ 
7      $u \leftarrow \text{Extract-Min}(Q)$ 
8     if  $u = t$ :
9       return Reconstruct( $\pi, t$ )
10     $S \leftarrow S \cup \{u\}$ 
11    for each  $v$  adjacent to  $u$ 
12      if  $v \in S$ : continue
13      if  $g[u] + w(u, v) < g[v]$ 
14         $g[v] \leftarrow g[u] + w(u, v)$ ,  $\pi[v] \leftarrow u$ 
15      Insert-or-Decrease-Key ( $Q, v, g[v]$ )
```

How Dijkstra picks

- Start from the source
- In each iteration, Picks the **locally best** node so far
- Explores paths for that node

Dhaka to Sylhet (using Dijkstra's algorithm)



How Dijkstra picks

- In each step, looks in every direction
- Picks the **locally best** node
- Explores paths for that node

Iteration 1

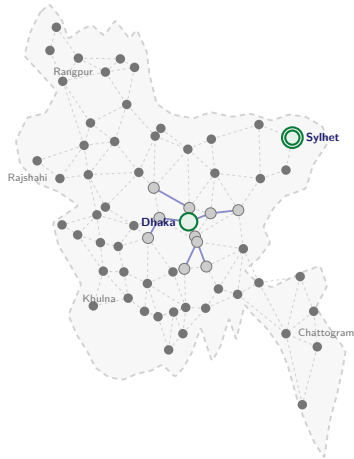
Iteration 2

Iteration 3

Iteration 4

Iteration 5

Dhaka to Sylhet (using Dijkstra's algorithm)



How Dijkstra picks

- In each step, looks in every direction
- Picks the **locally best** node
- Explores paths for that node

► Iteration 1

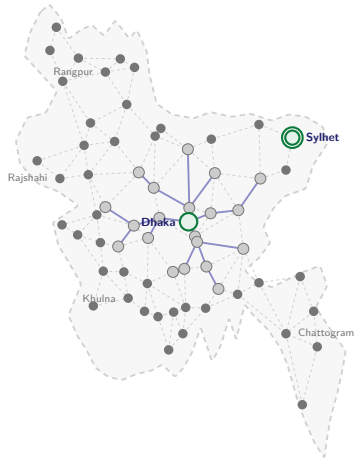
Iteration 2

Iteration 3

Iteration 4

Iteration 5

Dhaka to Sylhet (using Dijkstra's algorithm)



How Dijkstra picks

- In each step, looks in every direction
- Picks the **locally best** node
- Explores paths for that node

Iteration 1

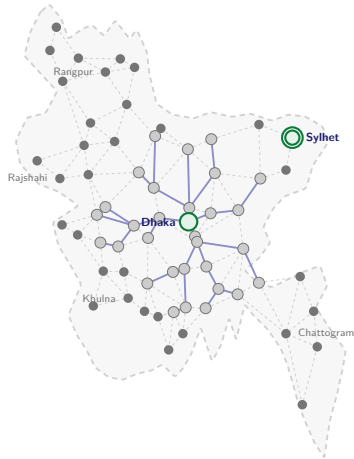
► **Iteration 2**

Iteration 3

Iteration 4

Iteration 5

Dhaka to Sylhet (using Dijkstra's algorithm)



How Dijkstra picks

- In each step, looks in every direction
- Picks the **locally best** node
- Explores paths for that node

Iteration 1

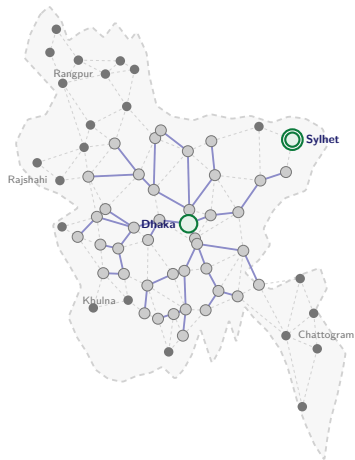
Iteration 2

► **Iteration 3**

Iteration 4

Iteration 5

Dhaka to Sylhet (using Dijkstra's algorithm)



How Dijkstra picks

- In each step, looks in every direction
- Picks the **locally best** node
- Explores paths for that node

Iteration 1

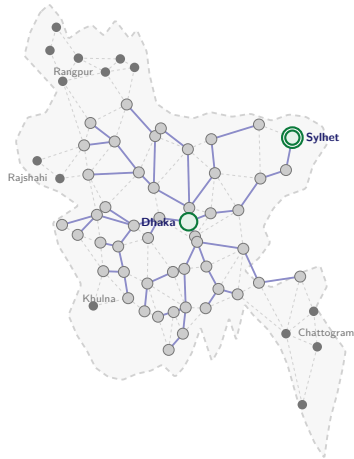
Iteration 2

Iteration 3

► **Iteration 4**

Iteration 5

Dhaka to Sylhet (using Dijkstra's algorithm)



How Dijkstra picks

- In each step, looks in every direction
- Picks the **locally best** node
- Explores paths for that node

Iteration 1

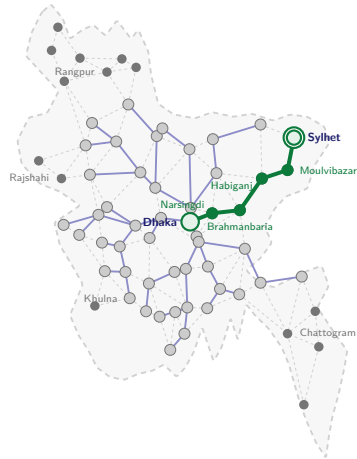
Iteration 2

Iteration 3

Iteration 4

► **Iteration 5**

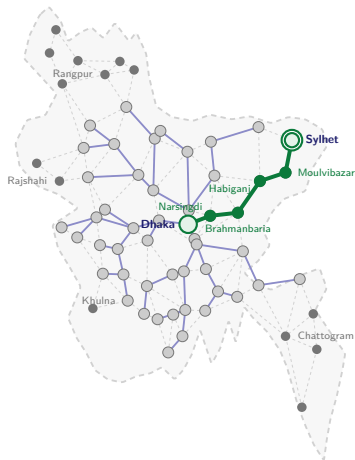
Dhaka to Sylhet (using Dijkstra's algorithm)



Path found



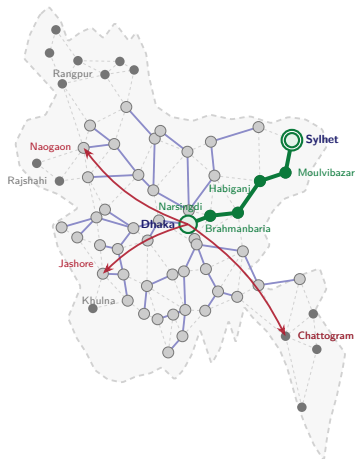
Dhaka to Sylhet — what the search cost



Result

Districts on the route	6 of 64
Districts explored	50 of 64
Optimal path?	Yes

Dhaka to Sylhet — what the search cost

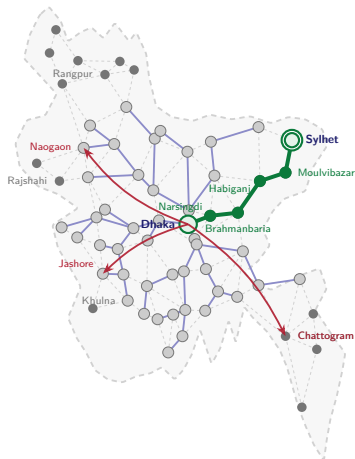


Result

Districts on the route	6 of 64
Districts explored	50 of 64
Optimal path?	Yes

- Dijkstra explores **in every direction**.

Dhaka to Sylhet — what the search cost



Result

Districts on the route	6 of 64
Districts explored	50 of 64
Optimal path?	Yes

- Dijkstra explores **in every direction**.
- We can solve this using a **greedy heuristic**.

```

1 Greedy-Best-First( $G, s, t$ )
2   for each  $v \in V$ :  $\pi[v] \leftarrow \text{nil}$ 
3    $S \leftarrow \{s\}$ 
4    $Q \leftarrow$  priority queue keyed by  $h$ , holding  $\{s\}$ 
5   while  $Q \neq \emptyset$ 
6      $u \leftarrow \text{Extract-Min}(Q)$ 
7     if  $u = t$ :
8       return  $\text{Reconstruct}(\pi, t)$ 
9     for each  $v$  adjacent to  $u$ 
10      if  $v \in S$ : continue
11       $S \leftarrow S \cup \{v\}$ ,  $\pi[v] \leftarrow u$ 
12       $\text{Insert}(Q, v, h(v))$ 

```



```
1 Greedy-Best-First( $G, s, t$ )
2   for each  $v \in V$ :  $\pi[v] \leftarrow \text{nil}$ 
3    $S \leftarrow \{s\}$ 
4    $Q \leftarrow$  priority queue keyed by  $h$ , holding  $\{s\}$ 
5   while  $Q \neq \emptyset$ 
6      $u \leftarrow \text{Extract-Min}(Q)$ 
7     if  $u = t$ :
8       return Reconstruct( $\pi, t$ )
9     for each  $v$  adjacent to  $u$ 
10      if  $v \in S$ : continue
11       $S \leftarrow S \cup \{v\}$ ,  $\pi[v] \leftarrow u$ 
12      Insert( $Q, v, h(v)$ )
```

How Greedy picks



```
1 Greedy-Best-First( $G, s, t$ )
2   for each  $v \in V$ :  $\pi[v] \leftarrow \text{nil}$ 
3    $S \leftarrow \{s\}$ 
4    $Q \leftarrow$  priority queue keyed by  $h$ , holding  $\{s\}$ 
5   while  $Q \neq \emptyset$ 
6      $u \leftarrow \text{Extract-Min}(Q)$ 
7     if  $u = t$ :
8       return Reconstruct( $\pi, t$ )
9     for each  $v$  adjacent to  $u$ 
10      if  $v \in S$ : continue
11       $S \leftarrow S \cup \{v\}$ ,  $\pi[v] \leftarrow u$ 
12      Insert( $Q, v, h(v)$ )
```

How Greedy picks

- In each step, looks in every direction



```
1 Greedy-Best-First( $G, s, t$ )
2   for each  $v \in V$ :  $\pi[v] \leftarrow \text{nil}$ 
3    $S \leftarrow \{s\}$ 
4    $Q \leftarrow$  priority queue keyed by  $h$ , holding  $\{s\}$ 
5   while  $Q \neq \emptyset$ 
6      $u \leftarrow \text{Extract-Min}(Q)$ 
7     if  $u = t$ :
8       return Reconstruct( $\pi, t$ )
9     for each  $v$  adjacent to  $u$ 
10      if  $v \in S$ : continue
11       $S \leftarrow S \cup \{v\}$ ,  $\pi[v] \leftarrow u$ 
12      Insert( $Q, v, h(v)$ )
```

$h(v)$ is the **straight-line distance** from v to Sylhet — a guess, never a measured road.

How Greedy picks

- In each step, looks in every direction
- Picks the node with the smallest h — the one that *looks* closest to Sylhet



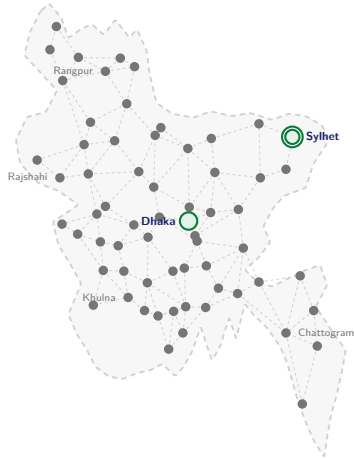
```
1 Greedy-Best-First( $G, s, t$ )
2   for each  $v \in V$ :  $\pi[v] \leftarrow \text{nil}$ 
3    $S \leftarrow \{s\}$ 
4    $Q \leftarrow$  priority queue keyed by  $h$ , holding  $\{s\}$ 
5   while  $Q \neq \emptyset$ 
6      $u \leftarrow \text{Extract-Min}(Q)$ 
7     if  $u = t$ :
8       return Reconstruct( $\pi, t$ )
9     for each  $v$  adjacent to  $u$ 
10      if  $v \in S$ : continue
11       $S \leftarrow S \cup \{v\}$ ,  $\pi[v] \leftarrow u$ 
12      Insert( $Q, v, h(v)$ )
```

$h(v)$ is the **straight-line distance** from v to Sylhet — a guess, never a measured road.

How Greedy picks

- In each step, looks in every direction
- Picks the node with the smallest h — the one that *looks* closest to Sylhet
- Explores paths for that node

Dhaka to Sylhet (using Greedy Best-First)

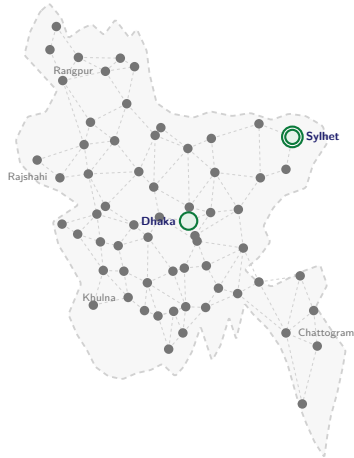


How Greedy picks

- In each step, looks in every direction
- Picks the node with the smallest h
- Explores paths for that node

Iteration 1
Iteration 2
Iteration 3
Iteration 4
Iteration 5
Iteration 6

Dhaka to Sylhet (using Greedy Best-First)



How Greedy picks

- In each step, looks in every direction
- Picks the node with the smallest h
- Explores paths for that node

► Iteration 1

Iteration 2

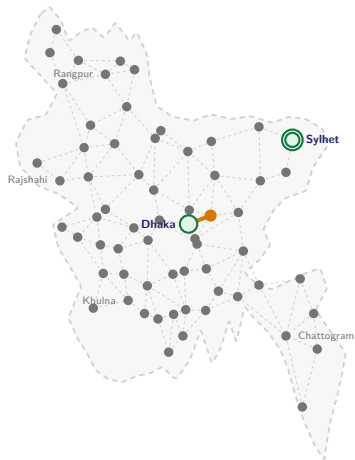
Iteration 3

Iteration 4

Iteration 5

Iteration 6

Dhaka to Sylhet (using Greedy Best-First)



How Greedy picks

- In each step, looks in every direction
- Picks the node with the smallest h
- Explores paths for that node

Iteration 1

► **Iteration 2**

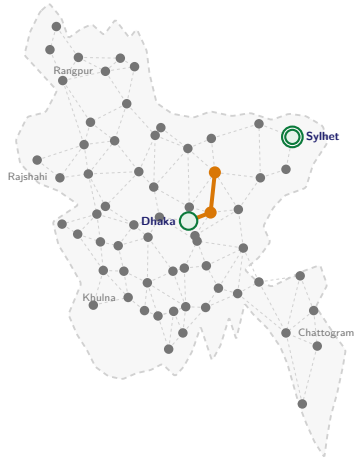
Iteration 3

Iteration 4

Iteration 5

Iteration 6

Dhaka to Sylhet (using Greedy Best-First)



How Greedy picks

- In each step, looks in every direction
- Picks the node with the smallest h
- Explores paths for that node

Iteration 1

Iteration 2

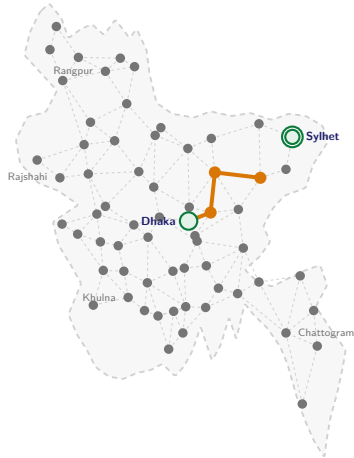
► **Iteration 3**

Iteration 4

Iteration 5

Iteration 6

Dhaka to Sylhet (using Greedy Best-First)



How Greedy picks

- In each step, looks in every direction
- Picks the node with the smallest h
- Explores paths for that node

Iteration 1

Iteration 2

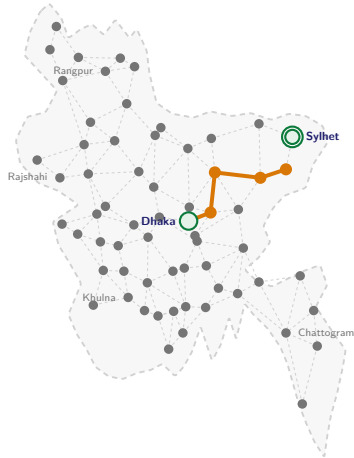
Iteration 3

► **Iteration 4**

Iteration 5

Iteration 6

Dhaka to Sylhet (using Greedy Best-First)



How Greedy picks

- In each step, looks in every direction
- Picks the node with the smallest h
- Explores paths for that node

Iteration 1

Iteration 2

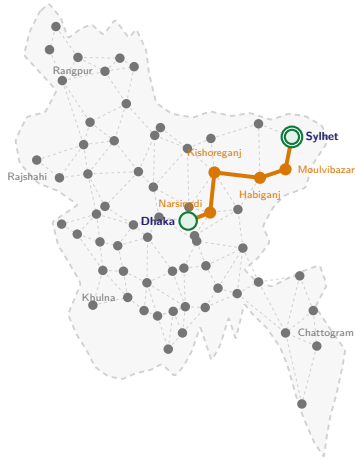
Iteration 3

Iteration 4

► **Iteration 5**

Iteration 6

Dhaka to Sylhet (using Greedy Best-First)



How Greedy picks

- In each step, looks in every direction
- Picks the node with the smallest h
- Explores paths for that node

Iteration 1

Iteration 2

Iteration 3

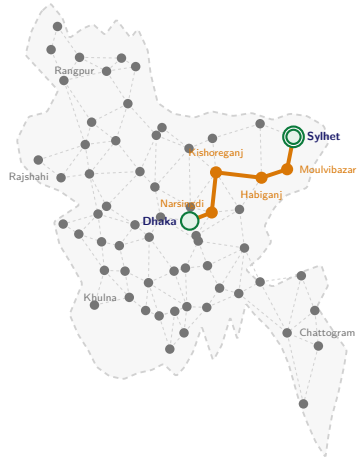
Iteration 4

Iteration 5

► Iteration 6

Six districts came off the queue.

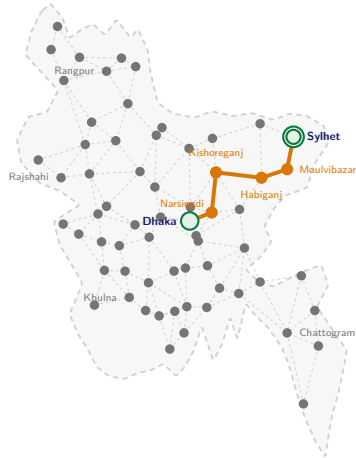
Greedy Best-First — the path and the price



Path found

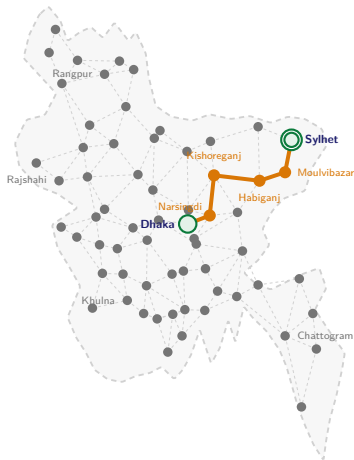


Greedy Best-First — the path and the price



Result	Greedy	Dijkstra
Districts explored	6 of 64	50 of 64
Route length	243 km	215 km
Optimal path?	No	Yes

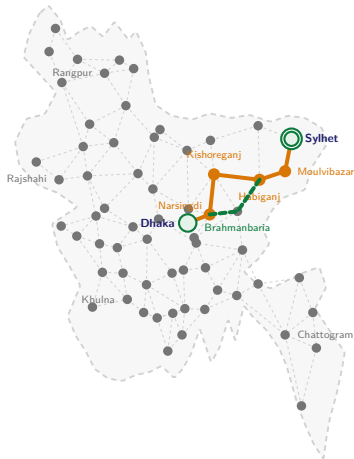
Greedy Best-First — the path and the price



Result	Greedy	Dijkstra
Districts explored	6 of 64	50 of 64
Route length	243 km	215 km
Optimal path?	No	Yes

- It's fast.

Greedy Best-First — the path and the price



— greedy, 243 km — cheapest, 215 km

Result	Greedy	Dijkstra
Districts explored	6 of 64	50 of 64
Route length	243 km	215 km
Optimal path?	No	Yes

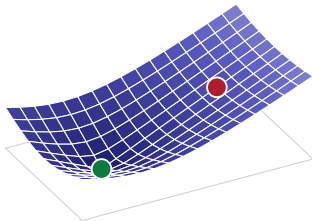
- It's fast.
- But it is not always optimal.



Dijkstra

ranks by g
the cost so far

✓ optimal ✗ not fast



Combining the two — A*



Dijkstra

ranks by g
the cost so far

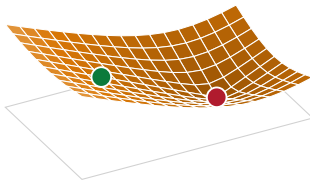
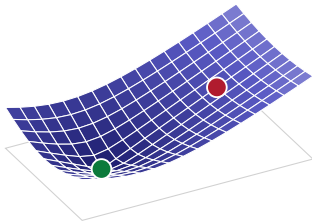
✓ optimal ✗ not fast



Greedy Best-First

ranks by h
a guess of what is left

✓ fast ✗ not optimal



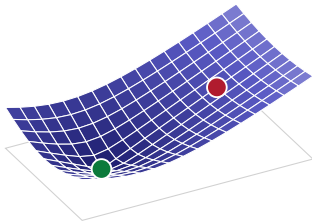
Combining the two — A*



Dijkstra

ranks by g
the cost so far

✓ optimal ✗ not fast

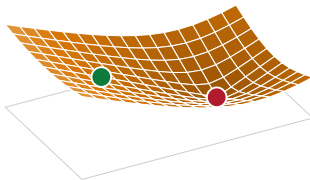


+

Greedy Best-First

ranks by h
a guess of what is left

✓ fast ✗ not optimal

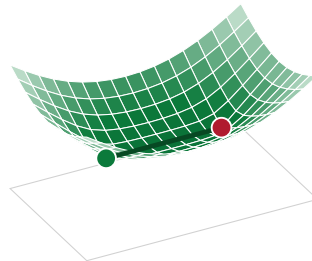


=

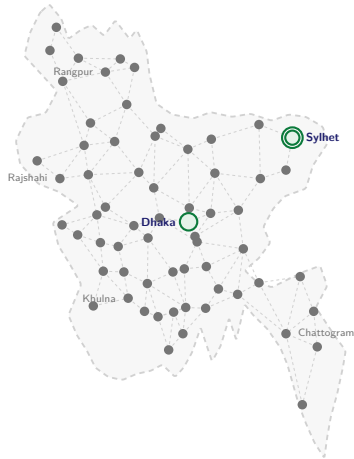
A*

ranks by $f = g + h$
the cost so far + the guess

✓ optimal ✓ fast



Dhaka to Sylhet (using A*)



How A* picks

- In each step, looks in every direction
- Picks the node with the smallest $f = g + h$
- Explores paths for that node

Iteration 1

Iteration 2

Iteration 3

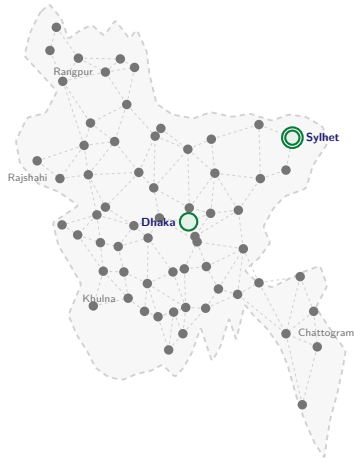
Iteration 4

Iteration 5

Iteration 6

Iteration 7

Iteration 8



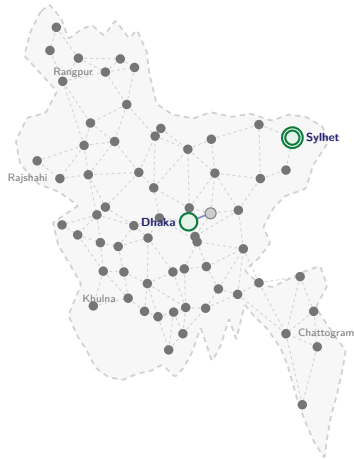
How A* picks

- In each step, looks in every direction
- Picks the node with the smallest $f = g + h$
- Explores paths for that node

► Iteration 1

Iteration 2
Iteration 3
Iteration 4
Iteration 5
Iteration 6
Iteration 7
Iteration 8

Dhaka to Sylhet (using A*)



How A* picks

- In each step, looks in every direction
- Picks the node with the smallest $f = g + h$
- Explores paths for that node

Iteration 1

► Iteration 2

Iteration 3

Iteration 4

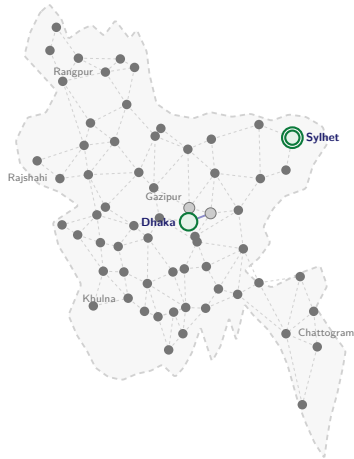
Iteration 5

Iteration 6

Iteration 7

Iteration 8

Dhaka to Sylhet (using A*)



How A* picks

- In each step, looks in every direction
- Picks the node with the smallest $f = g + h$
- Explores paths for that node

Iteration 1

Iteration 2

► **Iteration 3**

Iteration 4

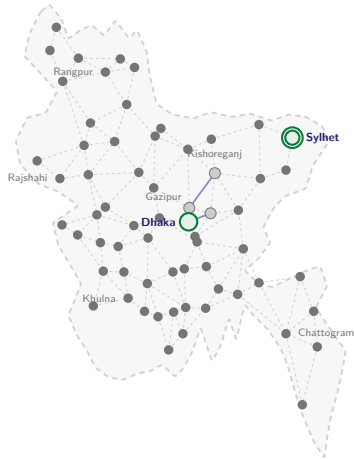
Iteration 5

Iteration 6

Iteration 7

Iteration 8

Dhaka to Sylhet (using A*)



How A* picks

- In each step, looks in every direction
- Picks the node with the smallest $f = g + h$
- Explores paths for that node

Iteration 1

Iteration 2

Iteration 3

► **Iteration 4**

Iteration 5

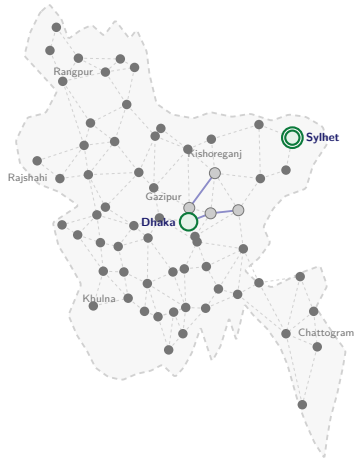
Iteration 6

Iteration 7

Iteration 8

Greedy's detour — A* looks, but g keeps it honest.

Dhaka to Sylhet (using A*)



How A* picks

- In each step, looks in every direction
- Picks the node with the smallest $f = g + h$
- Explores paths for that node

Iteration 1

Iteration 2

Iteration 3

Iteration 4

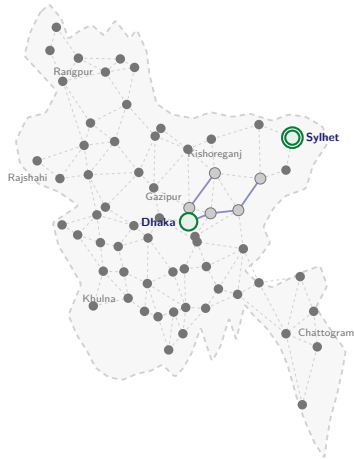
► **Iteration 5**

Iteration 6

Iteration 7

Iteration 8

Dhaka to Sylhet (using A*)



How A* picks

- In each step, looks in every direction
- Picks the node with the smallest $f = g + h$
- Explores paths for that node

Iteration 1

Iteration 2

Iteration 3

Iteration 4

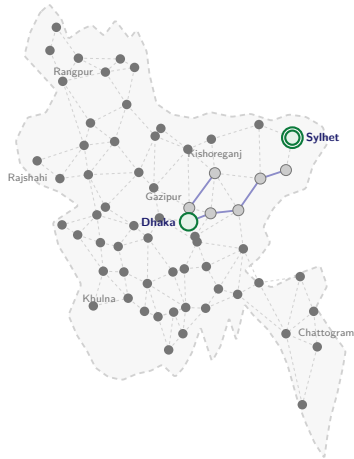
Iteration 5

► Iteration 6

Iteration 7

Iteration 8

Dhaka to Sylhet (using A*)



How A* picks

- In each step, looks in every direction
- Picks the node with the smallest $f = g + h$
- Explores paths for that node

Iteration 1

Iteration 2

Iteration 3

Iteration 4

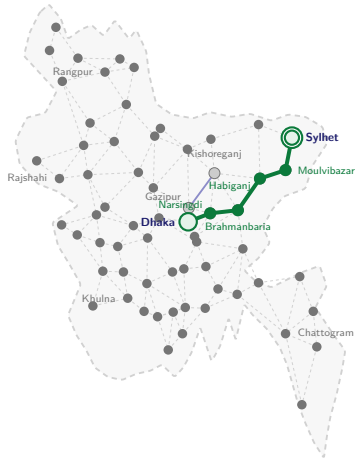
Iteration 5

Iteration 6

► **Iteration 7**

Iteration 8

Dhaka to Sylhet (using A*)



How A* picks

- In each step, looks in every direction
- Picks the node with the smallest $f = g + h$
- Explores paths for that node

Iteration 1

Iteration 2

Iteration 3

Iteration 4

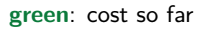
Iteration 5

Iteration 6

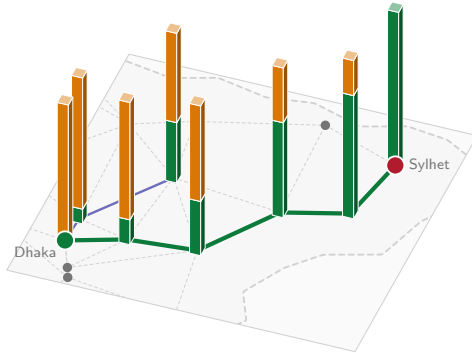
Iteration 7

► **Iteration 8**

Reached Sylhet. **Eight** districts came off the queue.



A* on the map — g hands off to h



green: cost so far

yellow: heuristic cost

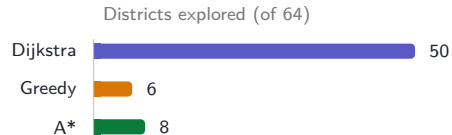


	■ Dijkstra	■ Greedy	■ A*
Districts explored	50 of 64	6 of 64	8 of 64
Route length	215 km	243 km	215 km
Optimal path?	✓ Yes	× No	✓ Yes

Dijkstra vs Greedy vs A*

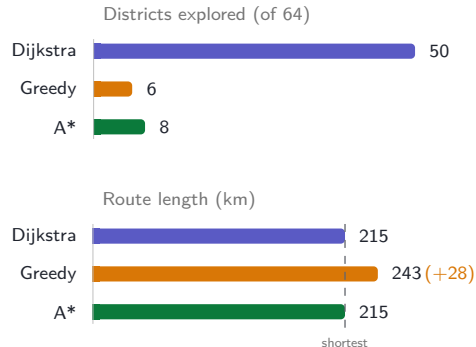


	■ Dijkstra	■ Greedy	■ A*
Districts explored	50 of 64	6 of 64	8 of 64
Route length	215 km	243 km	215 km
Optimal path?	✓ Yes	× No	✓ Yes

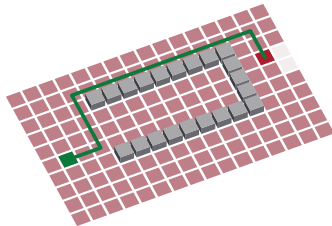




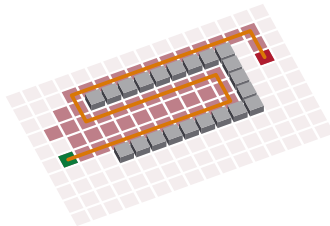
	■ Dijkstra	■ Greedy	■ A*
Districts explored	50 of 64	6 of 64	8 of 64
Route length	215 km	243 km	215 km
Optimal path?	✓ Yes	× No	✓ Yes



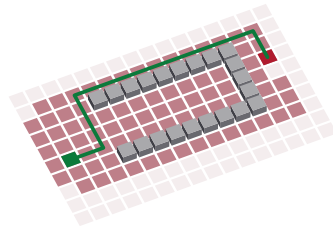
Summary — Dijkstra vs Greedy vs A*



Dijkstra
cells: 137
cost: 19

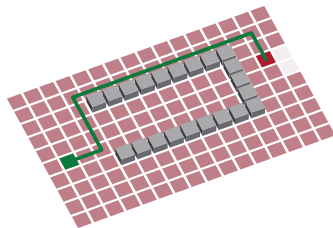


Greedy
cells: 48
cost: 35

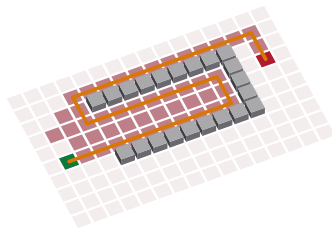


A*
cells: 81
cost: 19

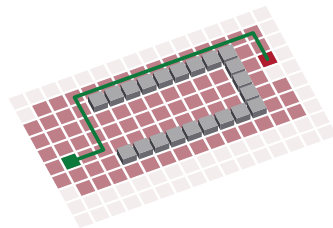
Summary — Dijkstra vs Greedy vs A*



Dijkstra
cells: 137
cost: 19

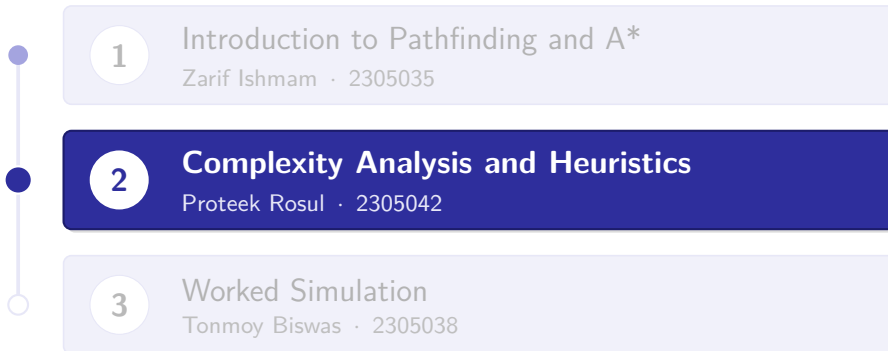


Greedy
cells: 48
cost: 35



A*
cells: 81
cost: 19

Far fewer expansions than Dijkstra, *and* a provably cheapest route — that is A*.







$g(v)$ — **cost so far**

The exact cost of the **best known path** from the start s to v .



$g(v)$ — **cost so far**

The exact cost of the **best known path** from the start s to v .

$h(v)$ — **cost to go**

An **estimate** of the remaining cost from v to the goal t .



$g(v)$ — **cost so far**

The exact cost of the **best known path** from the start s to v .

$h(v)$ — **cost to go**

An **estimate** of the remaining cost from v to the goal t .

$f(v) = g(v) + h(v)$ — **total estimated cost**

Best guess for the cost of the *full* $s \rightarrow t$ path if it routes through v .



$g(v)$ — **cost so far**

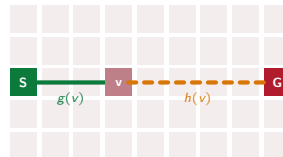
The exact cost of the **best known path** from the start s to v .

$h(v)$ — **cost to go**

An **estimate** of the remaining cost from v to the goal t .

$f(v) = g(v) + h(v)$ — **total estimated cost**

Best guess for the cost of the *full* $s \rightarrow t$ path if it routes through v .



$g(v) = 3$ — three steps walked

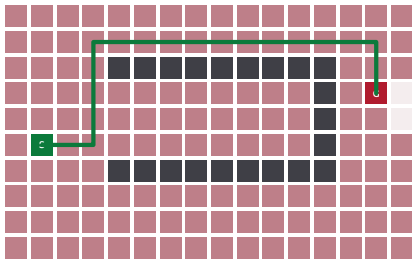
$h(v) = 5$ — five steps remaining

$f(v) = g(v) + h(v) = 3 + 5 = 8$

Why the heuristic helps



No heuristic — Dijkstra ($h \equiv 0$)

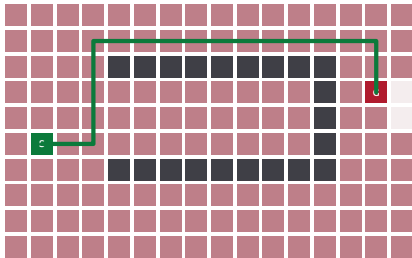


137 cells expanded — *blind* to where the goal is

Why the heuristic helps

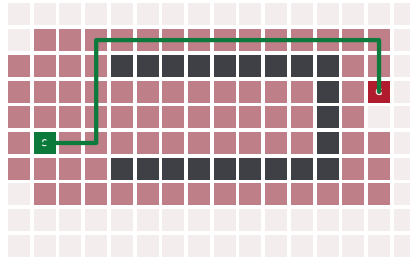


No heuristic — Dijkstra ($h \equiv 0$)



137 cells expanded — *blind* to where the goal is

With a good heuristic — A*

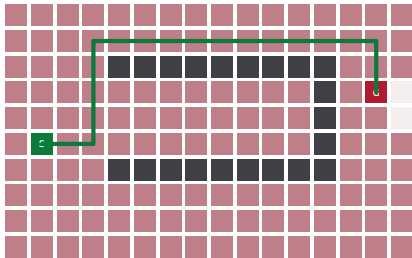


81 cells expanded — *biased* toward the goal

Why the heuristic helps

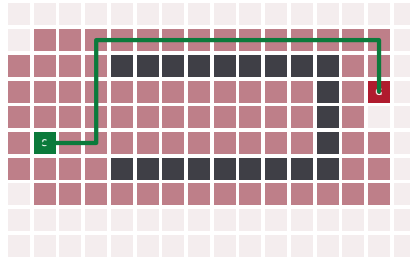


No heuristic — Dijkstra ($h \equiv 0$)



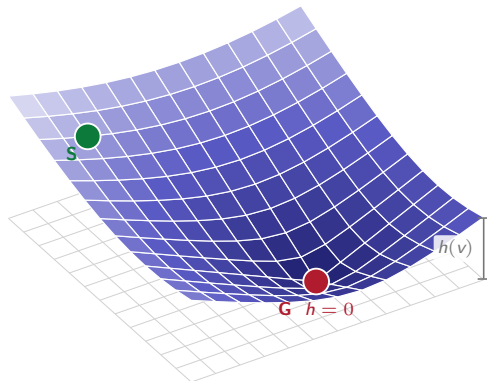
137 cells expanded — *blind* to where the goal is

With a good heuristic — A*



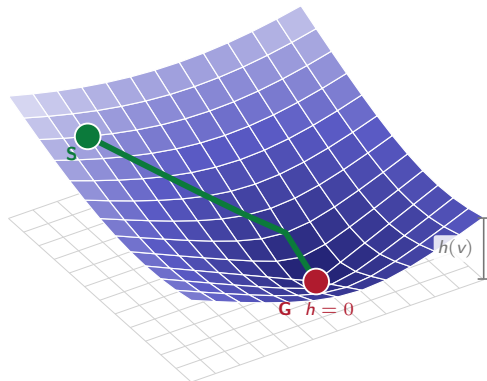
81 cells expanded — *biased* toward the goal

Takeaway. h does not change *what* A* is allowed to explore — it changes the *order*.



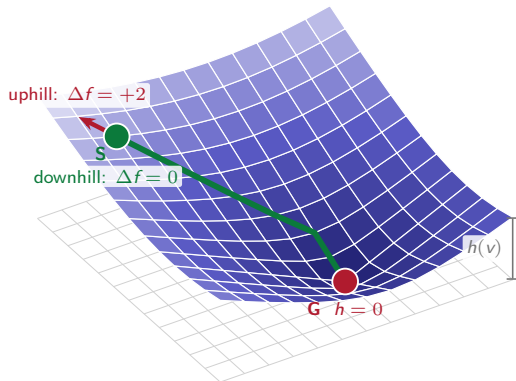
Lift the map by h

- raise every node v to height $h(v)$
- $h(G) = 0$



Lift the map by h

- raise every node v to height $h(v)$
- $h(G) = 0$
- each step costs $+1$ in g



Lift the map by h

- raise every node v to height $h(v)$
- $h(G) = 0$
- each step costs $+1$ in g
- a downhill step is incentivized
- an uphill step is penalized



What $h(v)$ has to be



What $h(v)$ has to be

- Cheap to compute



What $h(v)$ has to be

- Cheap to compute
- **Never an overestimate**



What $h(v)$ has to be

- Cheap to compute
- **Never an overestimate**
- Ideally close to the true remaining cost



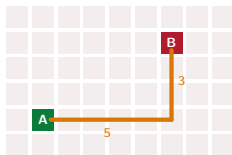
What $h(v)$ has to be

- Cheap to compute
- **Never an overestimate**
- Ideally close to the true remaining cost

Common choices

- Manhattan distance
- Euclidean distance

Manhattan vs Euclidean distance



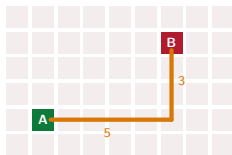
Manhattan

$$|x_A - x_B| + |y_A - y_B|$$

only horizontal / vertical steps —

$$5 + 3 = 8$$

Manhattan vs Euclidean distance

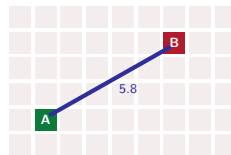


Manhattan

$$|x_A - x_B| + |y_A - y_B|$$

only horizontal / vertical steps —

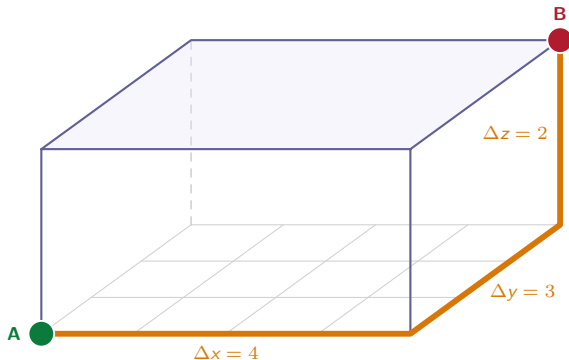
$$5 + 3 = 8$$



Euclidean

$$\sqrt{(x_A - x_B)^2 + (y_A - y_B)^2}$$

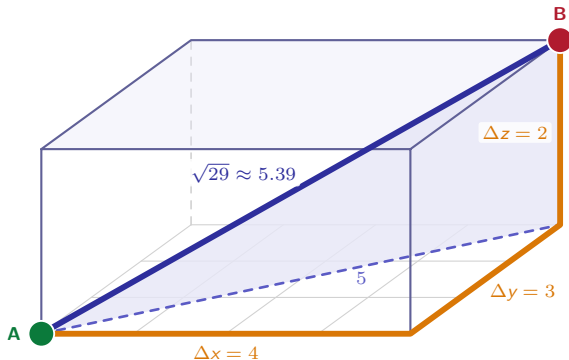
any-angle movement — $\sqrt{5^2 + 3^2} \approx 5.8$



Manhattan — along the edges

$$|\Delta x| + |\Delta y| + |\Delta z| = 4 + 3 + 2 = 9$$

one axis at a time, like a 6-directional grid



Manhattan — along the edges

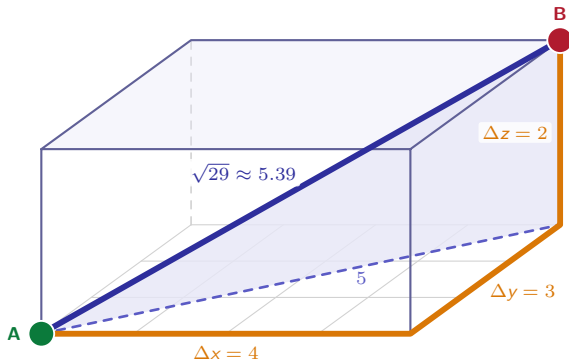
$$|\Delta x| + |\Delta y| + |\Delta z| = 4 + 3 + 2 = 9$$

one axis at a time, like a 6-directional grid

Euclidean — straight through

$$\sqrt{\Delta x^2 + \Delta y^2 + \Delta z^2} = \sqrt{29} \approx 5.39$$

any-angle movement



Manhattan — along the edges

$$|\Delta x| + |\Delta y| + |\Delta z| = 4 + 3 + 2 = 9$$

one axis at a time, like a 6-directional grid

Euclidean — straight through

$$\sqrt{\Delta x^2 + \Delta y^2 + \Delta z^2} = \sqrt{29} \approx 5.39$$

any-angle movement

Manhattan is **never smaller**, equal only when A and B differ along a single axis.



```
1 A-Star( $G, s, t, h$ )
2   for each  $v \in V$ :  $g[v] \leftarrow \infty$ ,  $\pi[v] \leftarrow \text{nil}$ 
3    $g[s] \leftarrow 0$ 
4    $Q \leftarrow$  priority queue keyed by  $g+h$ , holding  $\{s\}$ 
5    $S \leftarrow \emptyset$ 
6   while  $Q \neq \emptyset$ 
7      $u \leftarrow \text{Extract-Min}(Q)$ 
8     if  $u = t$ :
9       return Reconstruct( $\pi, t$ )
10     $S \leftarrow S \cup \{u\}$ 
11    for each  $v$  adjacent to  $u$ 
12      if  $v \in S$ : continue
13      if  $g[u] + w(u, v) < g[v]$ 
14         $g[v] \leftarrow g[u] + w(u, v)$ ,  $\pi[v] \leftarrow u$ 
15        Insert-or-Decrease-Key ( $Q, v, g[v] + h(v)$ )
```



```
1 A-Star( $G, s, t, h$ )
2   for each  $v \in V$ :  $g[v] \leftarrow \infty$ ,  $\pi[v] \leftarrow \text{nil}$ 
3    $g[s] \leftarrow 0$ 
4    $Q \leftarrow$  priority queue keyed by  $g+h$ , holding  $\{s\}$ 
5    $S \leftarrow \emptyset$ 
6   while  $Q \neq \emptyset$ 
7      $u \leftarrow \text{Extract-Min}(Q)$ 
8     if  $u = t$ :
9       return Reconstruct( $\pi, t$ )
10     $S \leftarrow S \cup \{u\}$ 
11    for each  $v$  adjacent to  $u$ 
12      if  $v \in S$ : continue
13      if  $g[u] + w(u, v) < g[v]$ 
14         $g[v] \leftarrow g[u] + w(u, v)$ ,  $\pi[v] \leftarrow u$ 
15        Insert-or-Decrease-Key ( $Q, v, g[v] + h(v)$ )
```

Only one difference from Dijkstra

The priority key changes from g to $(g + h)$.
Everywhere else, the algorithm is untouched.



```
1 A-Star( $G, s, t, h$ )
2   for each  $v \in V$ :  $g[v] \leftarrow \infty$ ,  $\pi[v] \leftarrow \text{nil}$ 
3    $g[s] \leftarrow 0$ 
4    $Q \leftarrow$  priority queue keyed by  $g + h$ , holding  $\{s\}$ 
5    $S \leftarrow \emptyset$ 
6   while  $Q \neq \emptyset$ 
7      $u \leftarrow \text{Extract-Min}(Q)$ 
8     if  $u = t$ :
9       return Reconstruct( $\pi, t$ )
10     $S \leftarrow S \cup \{u\}$ 
11    for each  $v$  adjacent to  $u$ 
12      if  $v \in S$ : continue
13      if  $g[u] + w(u, v) < g[v]$ 
14         $g[v] \leftarrow g[u] + w(u, v)$ ,  $\pi[v] \leftarrow u$ 
15        Insert-or-Decrease-Key ( $Q, v, g[v] + h(v)$ )
```



```
1 A-Star( $G, s, t, h$ )
2   for each  $v \in V$ :  $g[v] \leftarrow \infty, \pi[v] \leftarrow \text{nil}$ 
3    $g[s] \leftarrow 0$ 
4    $Q \leftarrow$  priority queue keyed by  $g + h$ , holding  $\{s\}$ 
5    $S \leftarrow \emptyset$ 
6   while  $Q \neq \emptyset$ 
7      $u \leftarrow \text{Extract-Min}(Q)$ 
8     if  $u = t$ :
9       return Reconstruct( $\pi, t$ )
10     $S \leftarrow S \cup \{u\}$ 
11    for each  $v$  adjacent to  $u$ 
12      if  $v \in S$ : continue
13      if  $g[u] + w(u, v) < g[v]$ 
14         $g[v] \leftarrow g[u] + w(u, v), \pi[v] \leftarrow u$ 
15        Insert-or-Decrease-Key ( $Q, v, g[v] + h(v)$ )
```

Init

$O(|V|)$ for touching every vertex once

Main loop

$O(|V| \log |V|)$ on a binary heap

Edge scan

$O(|E|)$ for considering every edge

Heap update

$O(\log |V|)$ each, up to $O(|E| \log |V|)$ total



- **Initialization:** $O(|V|)$



- **Initialization:** $O(|V|)$
- **Extractions:** $O(|V| \log |V|)$



- **Initialization:** $O(|V|)$
- **Extractions:** $O(|V| \log |V|)$
- **Edge scans:** $O(|E|)$



- **Initialization:** $O(|V|)$
- **Extractions:** $O(|V| \log |V|)$
- **Edge scans:** $O(|E|)$
- **Heap insert / decrease-key:** $O(|E| \log |V|)$



- **Initialization:** $O(|V|)$
- **Extractions:** $O(|V| \log |V|)$
- **Edge scans:** $O(|E|)$
- **Heap insert / decrease-key:** $O(|E| \log |V|)$

$$O(|V|) + O(|V| \log |V|) + O(|E|) + O(|E| \log |V|)$$



- **Initialization:** $O(|V|)$
- **Extractions:** $O(|V| \log |V|)$
- **Edge scans:** $O(|E|)$
- **Heap insert / decrease-key:** $O(|E| \log |V|)$

$$O(|V|) + O(|V| \log |V|) + O(|E|) + O(|E| \log |V|)$$

$$\Rightarrow \boxed{O((|V| + |E|) \log |V|)}$$



```
1 A-Star( $G, s, t, h$ )
2   for each  $v \in V$ :  $g[v] \leftarrow \infty$ ,  $\pi[v] \leftarrow \text{nil}$ 
3    $g[s] \leftarrow 0$ 
4    $Q \leftarrow$  priority queue keyed by  $g + h$ , holding  $\{s\}$ 
5    $S \leftarrow \emptyset$ 
6   while  $Q \neq \emptyset$ 
7      $u \leftarrow \text{Extract-Min}(Q)$ 
8     if  $u = t$ :
9       return Reconstruct( $\pi, t$ )
10     $S \leftarrow S \cup \{u\}$ 
11    for each  $v$  adjacent to  $u$ 
12      if  $v \in S$ : continue
13      if  $g[u] + w(u, v) < g[v]$ 
14         $g[v] \leftarrow g[u] + w(u, v)$ ,  $\pi[v] \leftarrow u$ 
15        Insert-or-Decrease-Key ( $Q, v, g[v] + h(v)$ )
```



```
1 A-Star( $G, s, t, h$ )
2   for each  $v \in V$ :  $g[v] \leftarrow \infty$ ,  $\pi[v] \leftarrow \text{nil}$ 
3    $g[s] \leftarrow 0$ 
4    $Q \leftarrow$  priority queue keyed by  $g + h$ , holding  $\{s\}$ 
5    $S \leftarrow \emptyset$ 
6   while  $Q \neq \emptyset$ 
7      $u \leftarrow \text{Extract-Min}(Q)$ 
8     if  $u = t$ :
9       return Reconstruct( $\pi, t$ )
10     $S \leftarrow S \cup \{u\}$ 
11    for each  $v$  adjacent to  $u$ 
12      if  $v \in S$ : continue
13      if  $g[u] + w(u, v) < g[v]$ 
14         $g[v] \leftarrow g[u] + w(u, v)$ ,  $\pi[v] \leftarrow u$ 
15        Insert-or-Decrease-Key ( $Q, v, g[v] + h(v)$ )
```

g and π arrays

One entry per vertex: $O(|V|)$

Frontier Q and closed set S

Every vertex is generated at most once: together $O(|V|)$.

Reconstructed path

$O(|V|)$ as typically $O(d) \ll |V|$



- $g[\cdot]$ and $\pi[\cdot]$: $O(|V|)$



- $g[\cdot]$ and $\pi[\cdot]$: $O(|V|)$
- Priority queue Q and closed set S : $O(|V|)$



- $g[\cdot]$ and $\pi[\cdot]$: $O(|V|)$
- Priority queue Q and closed set S : $O(|V|)$
- Path reconstruction: $O(|V|)$ worst case



- $g[\cdot]$ and $\pi[\cdot]$: $O(|V|)$
- Priority queue Q and closed set S : $O(|V|)$
- Path reconstruction: $O(|V|)$ worst case

$$O(|V|) + O(|V|) + O(|V|)$$



- $g[\cdot]$ and $\pi[\cdot]$: $O(|V|)$
- Priority queue Q and closed set S : $O(|V|)$
- Path reconstruction: $O(|V|)$ worst case

$$O(|V|) + O(|V|) + O(|V|)$$

$$\Rightarrow \boxed{O(|V|)}$$



- $g[\cdot]$ and $\pi[\cdot]$: $O(|V|)$
- Priority queue Q and closed set S : $O(|V|)$
- Path reconstruction: $O(|V|)$ worst case

$$O(|V|) + O(|V|) + O(|V|)$$

$$\Rightarrow \boxed{O(|V|)}$$

No dependence on $|E|$.



	Dijkstra	Greedy	A*
Time (worst case)	$O((V + E) \log V)$	$O((V + E) \log V)$	$O((V + E) \log V)$
Space	$O(V)$	$O(V)$	$O(V)$
Nodes expanded	most / all	few, unpredictable	few, provably safe
Optimal?	✓	✗	✓



	Dijkstra	Greedy	A*
Time (worst case)	$O((V + E) \log V)$	$O((V + E) \log V)$	$O((V + E) \log V)$
Space	$O(V)$	$O(V)$	$O(V)$
Nodes expanded	most / all	few, unpredictable	few, provably safe
Optimal?	✓	✗	✓

Same asymptotic worst case as Dijkstra — A* wins in *practice*, not in the textbook bound.



1
Admissibility



1

Admissibility

2

Consistency



Definition

A heuristic h is **admissible** if, for every node v ,

$$h(v) \leq h^*(v)$$

where $h^*(v)$ is the *true* cheapest cost from v to the goal.

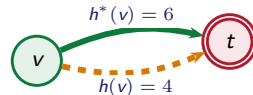


Definition

A heuristic h is **admissible** if, for every node v ,

$$h(v) \leq h^*(v)$$

where $h^*(v)$ is the *true* cheapest cost from v to the goal.



$h(v) = 4 \leq h^*(v) = 6$ — admissible

if instead $h(v) = 8$ — not admissible

Why it guarantees optimality

When A* pops t , every other node in Q has $f = g + h \geq$ that f -value. Since h never overestimates, nothing left in the queue can beat t 's true cost.



Definition

A heuristic h is **consistent** if, for every edge (u, v) with weight $w(u, v)$,

$$h(u) \leq w(u, v) + h(v)$$

with $h(\text{goal}) = 0$.



Definition

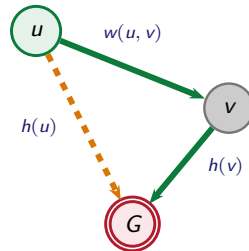
A heuristic h is **consistent** if, for every edge (u, v) with weight $w(u, v)$,

$$h(u) \leq w(u, v) + h(v)$$

with $h(\text{goal}) = 0$.

Why it matters

The **triangle inequality** applied to h :
 $f = g + h$ never decreases along a path.

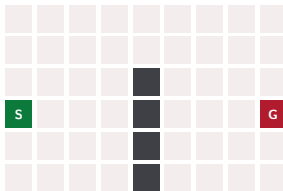


$h(u) \leq w(u, v) + h(v)$ — the triangle inequality in effect.

A bad heuristic (I) — overestimating



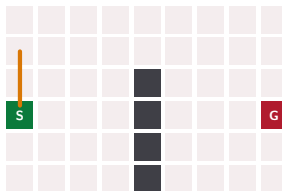
Overestimating h — looks like Greedy



A bad heuristic (I) — overestimating



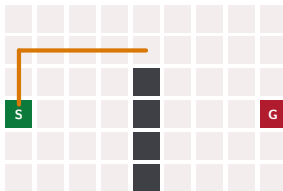
Overestimating h — looks like Greedy



A bad heuristic (I) — overestimating



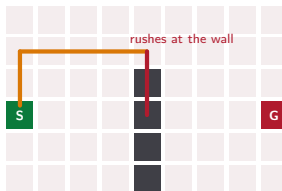
Overestimating h — looks like Greedy



A bad heuristic (I) — overestimating



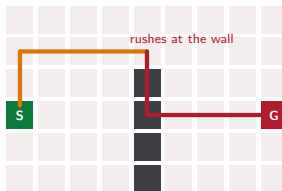
Overestimating h — looks like Greedy



A bad heuristic (I) — overestimating



Overestimating h — looks like Greedy

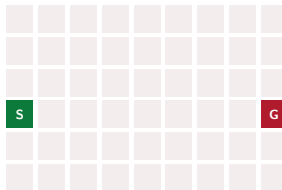


- Inadmissible
- Behaves like Greedy
- Can return a longer-than-optimal path

A bad heuristic (II) — uninformative



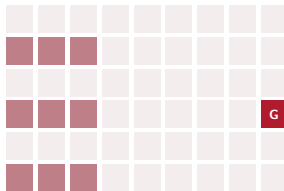
Uninformative h — looks like Dijkstra



A bad heuristic (II) — uninformative



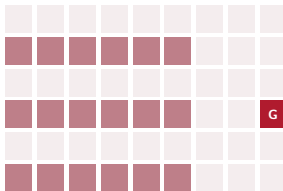
Uninformative h — looks like Dijkstra



A bad heuristic (II) — uninformative

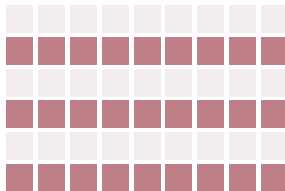


Uninformative h — looks like Dijkstra





Uninformative h — looks like Dijkstra

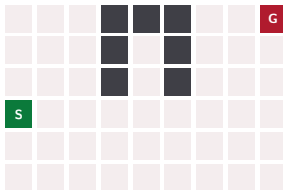


- $h(v) = 1$ for all non-goal v
- **Admissible**, but slow — same as Dijkstra

A bad heuristic (III) — admissible but misleading



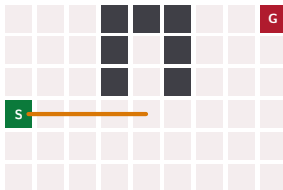
Admissible h , dead-end pocket — correct, yet still wasteful



A bad heuristic (III) — admissible but misleading



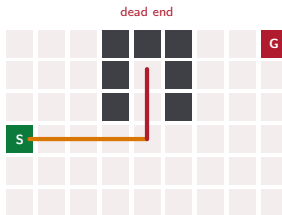
Admissible h , dead-end pocket — correct, yet still wasteful



A bad heuristic (III) — admissible but misleading



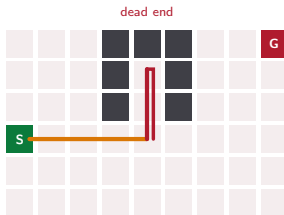
Admissible h , dead-end pocket — correct, yet still wasteful



A bad heuristic (III) — admissible but misleading



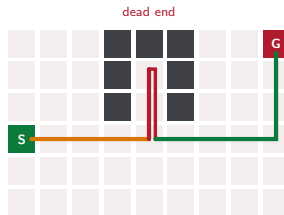
Admissible h , dead-end pocket — correct, yet still wasteful



A bad heuristic (III) — admissible but misleading



Admissible h , dead-end pocket — correct, yet still wasteful



- Still **admissible** — just blind to the obstacle
- Finds the optimal path, but wastes a detour on a dead end

Three ways a heuristic can go wrong



Three ways a heuristic can go wrong



Too large

inadmissible — overestimates
the remaining cost

✗ correctness ✓ decisive

Three ways a heuristic can go wrong



Too large

inadmissible — overestimates
the remaining cost

✗ correctness ✓ decisive

Too flat

uninformative — admissible
but barely varies

✓ correct ✗ degrades to Dijkstra

Three ways a heuristic can go wrong



Too large

inadmissible — overestimates
the remaining cost

✗ correctness ✓ decisive

Too flat

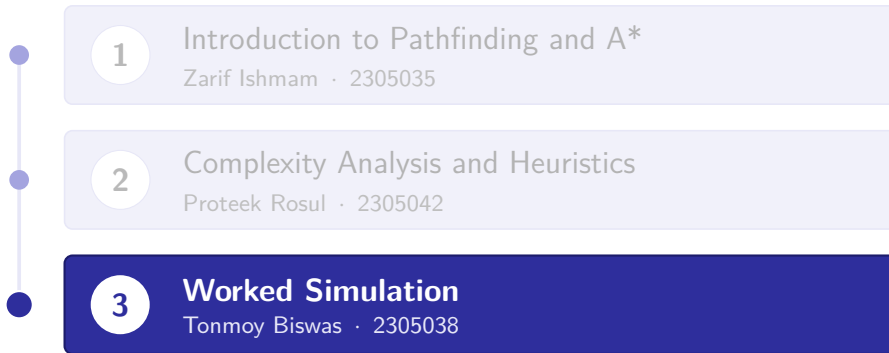
uninformative — admissible
but barely varies

✓ correct ✗ degrades to Dijkstra

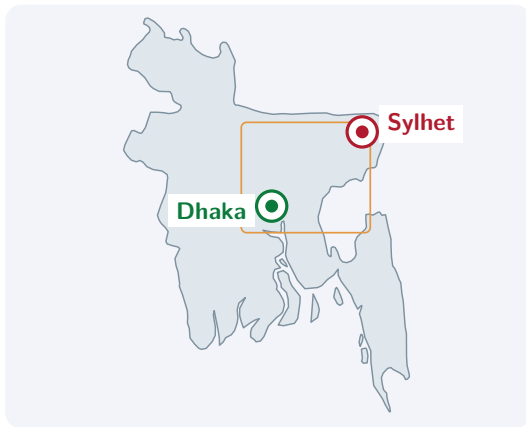
Blind to structure

(admissible, locally misleading) —
ignores walls and dead ends

✓ correct ✗ wastes expansions



We know A^* . Now, a trip to Sylhet.



A JOURNEY USING A^*

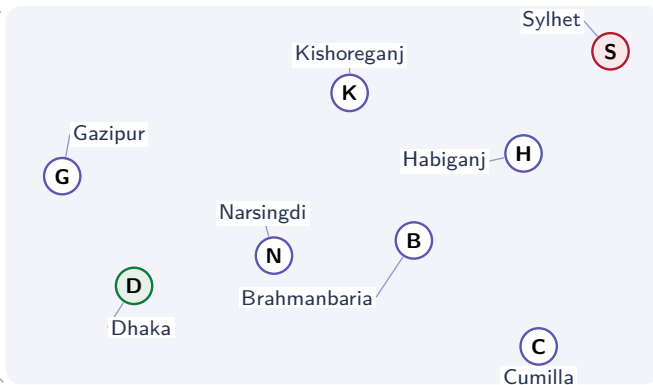
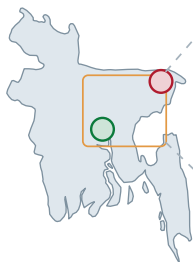
Dhaka to Sylhet

A journey. A destination.

Which route should we choose?



Dhaka to Sylhet

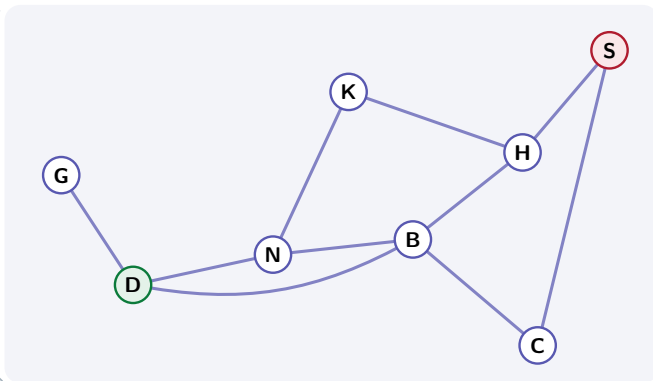
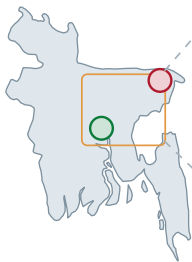


Eight cities. Each city is a node.

We know A*. Now, a trip to Sylhet.



Dhaka to Sylhet

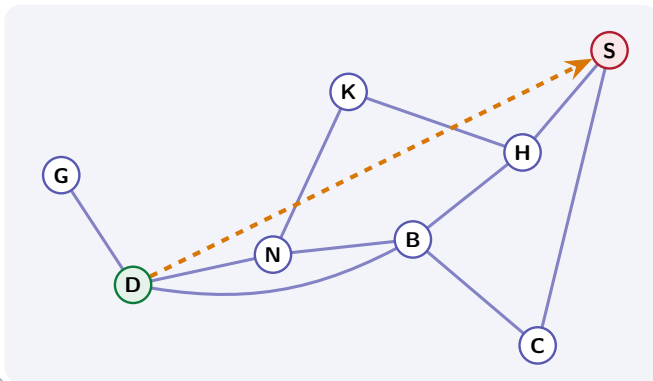
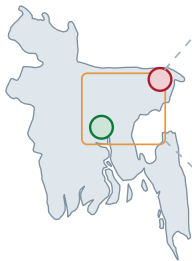


Ten roads. Each connection is an edge.

We know A*. Now, a trip to Sylhet.

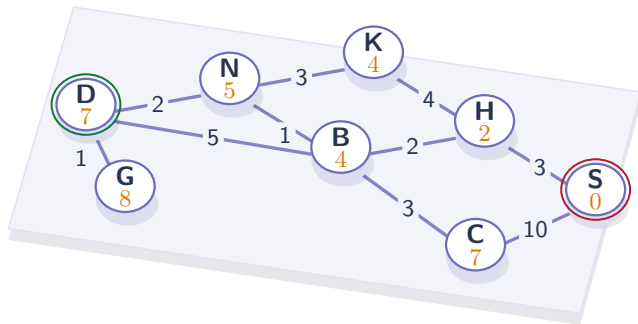


Dhaka to Sylhet



Straight-line (Euclidean) distance: a possible estimate.

We know A*. Now, a trip to Sylhet.



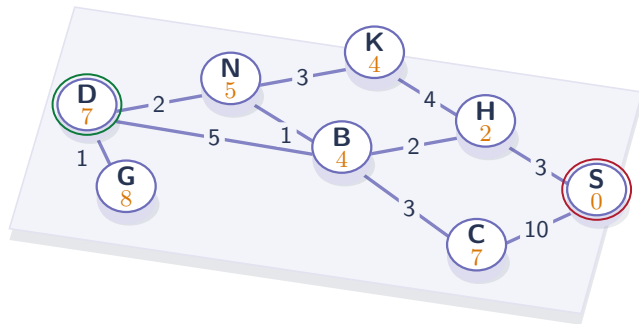
Now use a simple model

Orange number = h_cost .

h stays fixed; g and f change.

At the goal, $h(S) = 0$.

Starting the journey from D



OPEN smallest f first

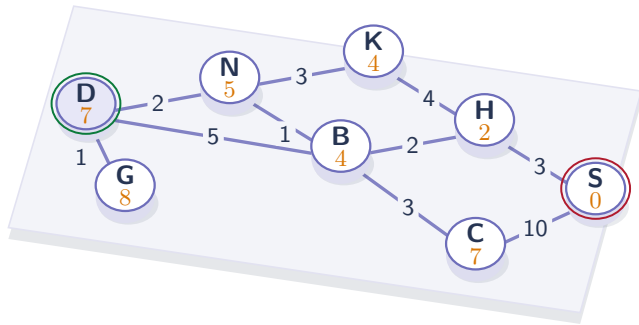
Current: —

Node	g	h	f
OPEN is empty			

Algorithm step

- 1 for $v \in V$: $g[v] \leftarrow \infty$
- 2 $g[s] \leftarrow 0$; $Q \leftarrow \{s\}$; $C \leftarrow \emptyset$

Only D enters OPEN



OPEN smallest f first

Current: —

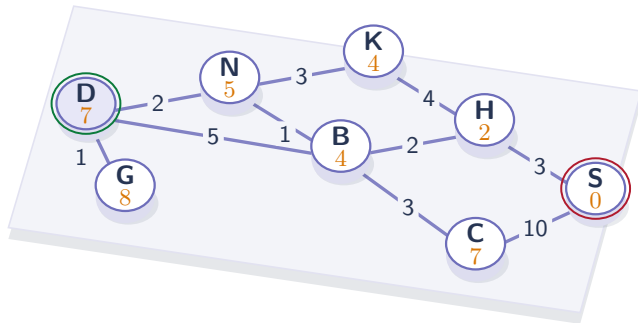
Node	g	h	f
D	0	7	7

$$g(D) = 0, \quad h(D) = 7, \quad f(D) = 7$$

Algorithm step

2 $g[s] \leftarrow 0; Q \leftarrow \{s\}; C \leftarrow \emptyset$

D is the only node in OPEN



OPEN smallest f first

Current: —

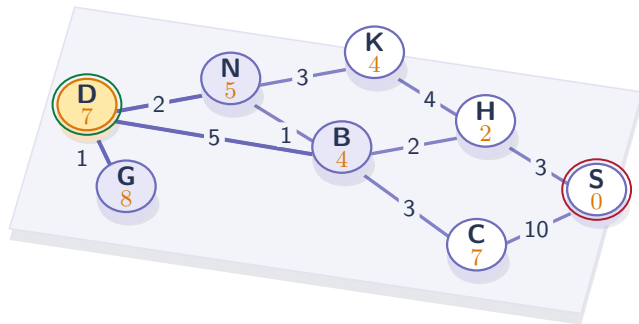
Node	g	h	f
D	0	7	7

A* picks D .

Algorithm step

```
4  $u \leftarrow \text{EXTRACT-MIN}(Q)$ 
5 if  $u = t$ : return  $\text{PATH}(t)$ 
6  $C \leftarrow C \cup \{u\}$ 
```

D's neighbors enter OPEN



OPEN smallest f first

Current: D

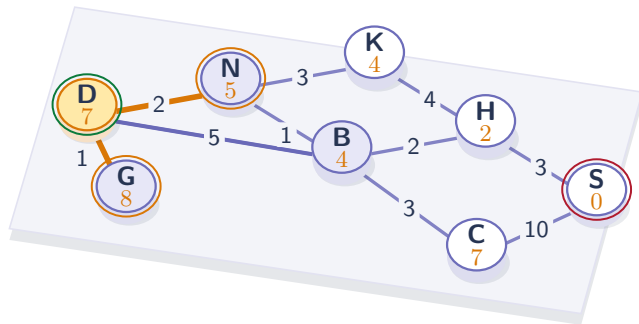
Node	g	h	f
N	2	5	7
B	5	4	9
G	1	8	9

Parent: $N \leftarrow D$, $B \leftarrow D$, $G \leftarrow D$

Algorithm step

```
10  $g[v] \leftarrow q$ ;  $\pi[v] \leftarrow u$   
11 UPDATE( $Q, v$ )
```

Why not the cheap road?



OPEN smallest f first

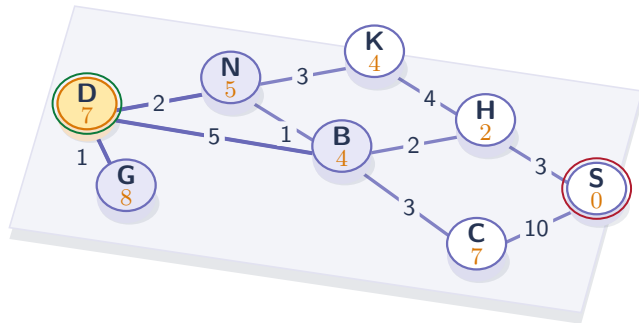
Current: D

Node	g	h	f
N	2	5	7
B	5	4	9
G	1	8	9

$$f(N) = 7 < f(G) = 9$$

Algorithm step

4 $u \leftarrow \text{EXTRACT-MIN}(Q)$



OPEN smallest f first

Current: D

Node	g	h	f
N	2	5	7
B	5	4	9
G	1	8	9

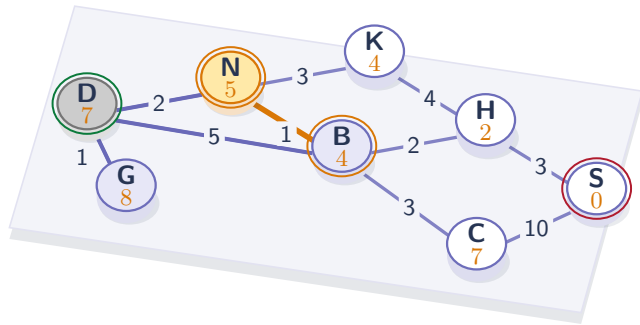
Select *N* next.

Algorithm step

```

4   $u \leftarrow \text{EXTRACT-MIN}(Q)$ 
5  if  $u = t$ : return PATH( $t$ )
6   $C \leftarrow C \cup \{u\}$ 
    
```

A cheaper way to B



OPEN smallest f first

Current: N

Node	g	h	f
B	5	4	9
G	1	8	9

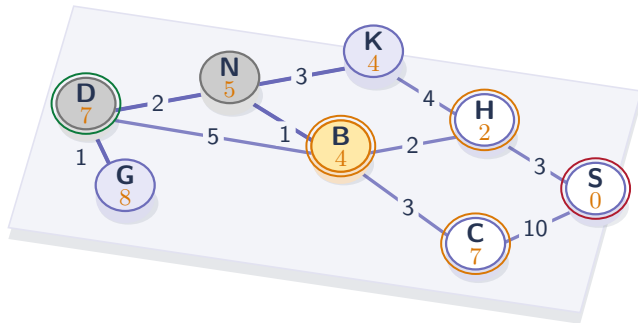
$$g_{\text{new}}(B) = 2 + 1 = 3 < 5$$

$$f_{\text{new}}(B) = 3 + 4 = 7$$

Algorithm step

8 $q \leftarrow g[u] + w(u, v)$

9 if $q < g[v]$



OPEN smallest f first

Current: B

Node	g	h	f
K	5	4	9
G	1	8	9

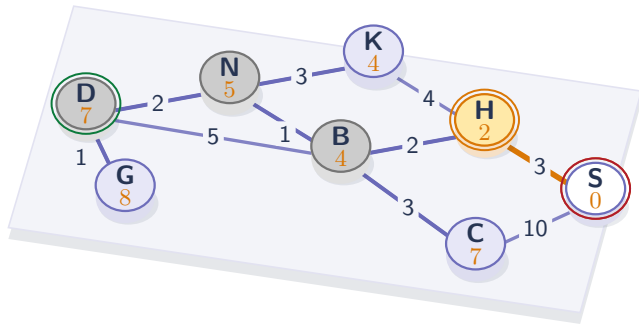
$u = B$; inspect H and C

Algorithm step

```

4   $u \leftarrow \text{EXTRACT-MIN}(Q)$ 
7  for  $v \in \text{Adj}[u] \setminus C$ 
    
```

The first route to S



OPEN smallest f first

Current: H

Node	g	h	f
K	5	4	9
G	1	8	9
C	6	7	13

$$g_{new}(S) = 5 + 3 = 8 < \infty$$

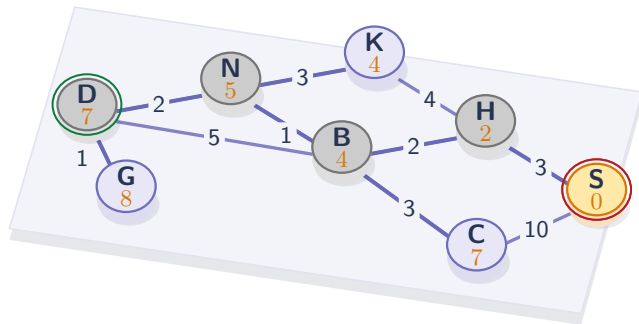
$$f_{new}(S) = 8 + 0 = 8$$

Algorithm step

8 $q \leftarrow g[u] + w(u, v)$

9 if $q < g[v]$

Now the search can stop



OPEN smallest f first

Current: S

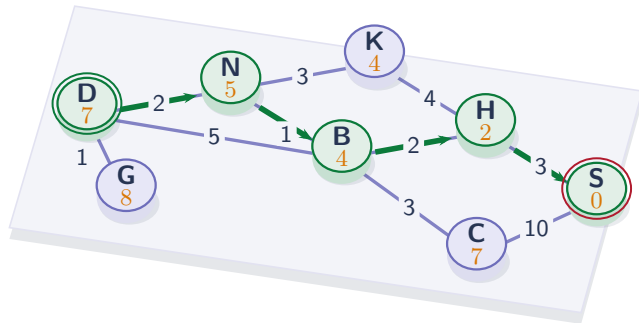
Node	g	h	f
K	5	4	9
G	1	8	9
C	6	7	13

$$u = S, \quad g(S) = 8$$

Algorithm step

5 if $u = t$: return PATH(t)

Follow the parents and reverse the list



OPEN smallest f first

Current: S

Node	g	h	f
K	5	4	9
G	1	8	9
C	6	7	13

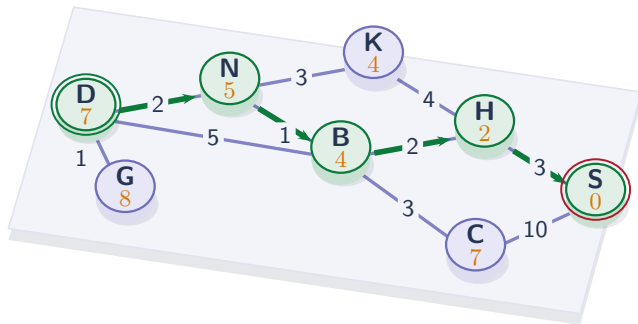
$D \rightarrow N \rightarrow B \rightarrow H \rightarrow S$

Cost: 8

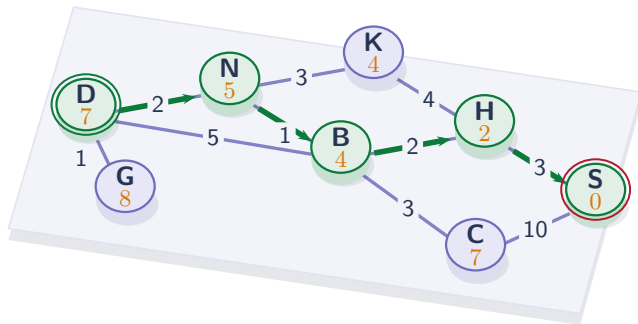
Algorithm step

5 if $u = t$: return $\text{PATH}(t)$

A new question about the heuristic



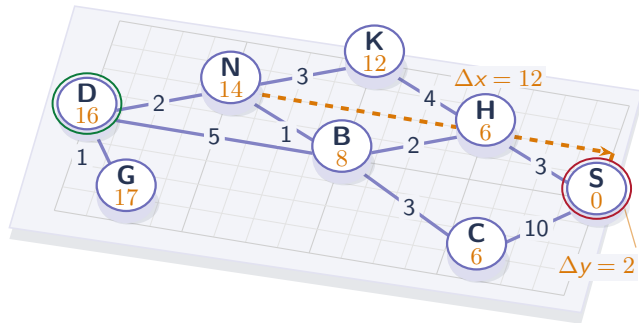
Will any heuristic function work here?



Let's try Manhattan distance.

Horizontal + vertical distance.

Can a different estimate change the answer?



Build the estimate

Horizontal $\Delta x = 12$

Vertical $\Delta y = 2$

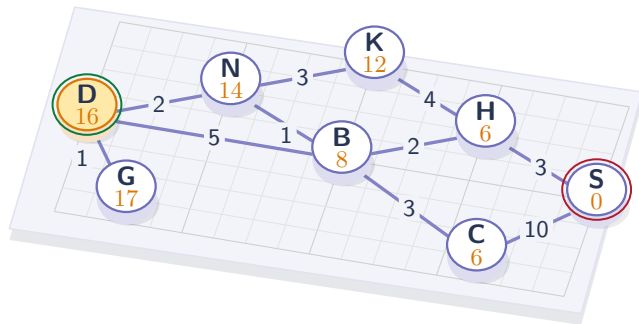
Total estimate

$$h_M(N) = 12 + 2 = 14$$

Schematic grid units;
not measured road costs.

Known best: 8 This run: —

Can a different estimate change the answer?



Restart: select D

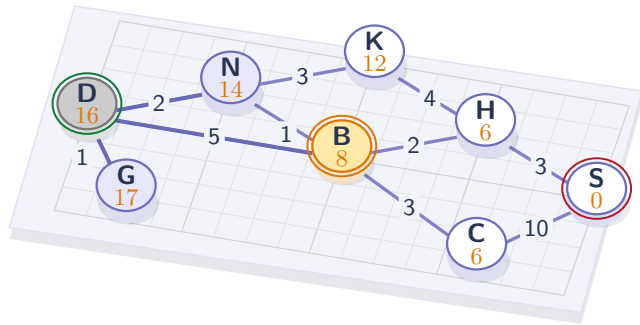
Node	g	h	f
D	0	16	16

$$g(D) = 0, \quad f(D) = 16$$

OPEN: smallest f first

Known best: 8 This run: —

Can a different estimate change the answer?



Select B

Node	g	h	f
N	2	14	16
G	1	17	18

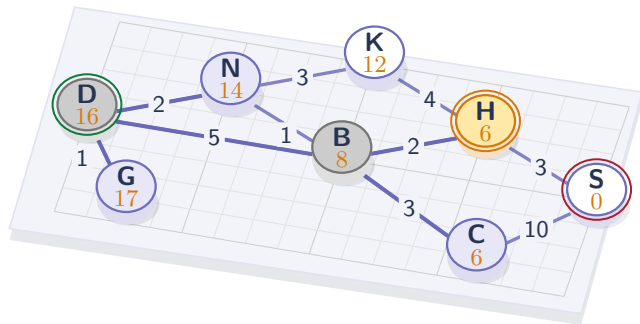
$$g(H) = 5 + 2 = 7$$

$$f(H) = 7 + 6 = 13$$

OPEN: smallest f first

Known best: 8 This run: —

Can a different estimate change the answer?



Select H

Node	g	h	f
C	8	6	14
N	2	14	16
G	1	17	18

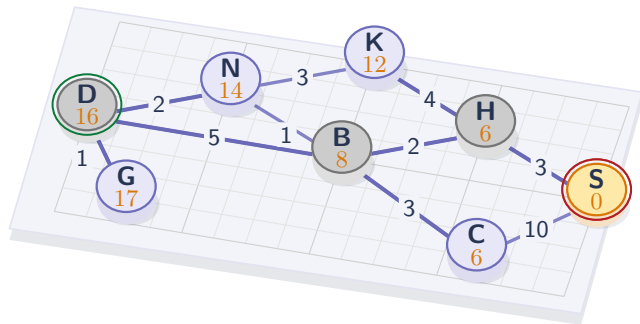
$$g(S) = 7 + 3 = 10$$

$$f(S) = 10 + 0 = 10$$

OPEN: smallest f first

Known best: 8 This run: —

Can a different estimate change the answer?



Extract S

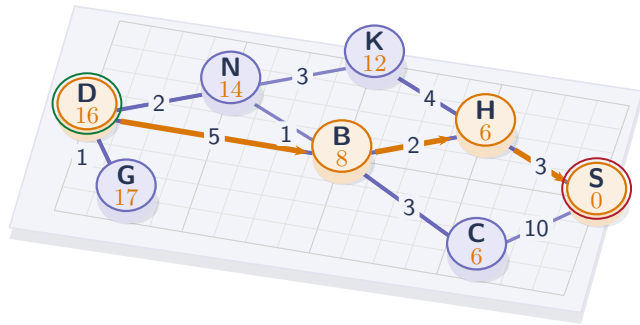
Node	g	h	f
C	8	6	14
N	2	14	16
G	1	17	18
K	11	12	23

$$u = S, \quad g(S) = 10$$

OPEN: smallest f first

Known best: 8 This run: 10

We didn't get the shortest path



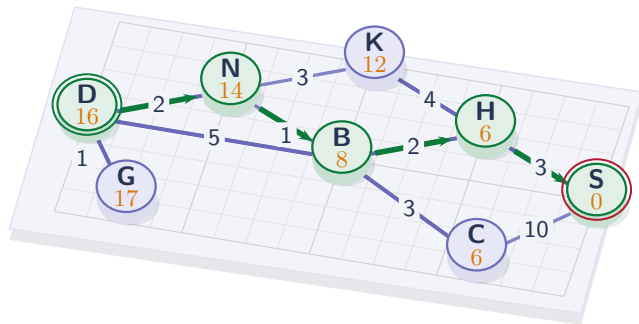
Returned route

$D \rightarrow B \rightarrow H \rightarrow S$

This run: 10

Known best: 8 This run: 10

A cheaper route was still there



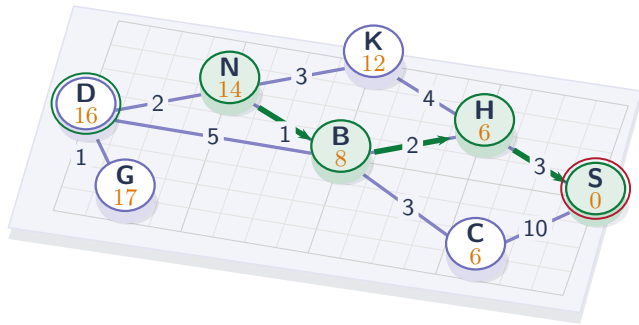
Optimal route

$D \rightarrow N \rightarrow B \rightarrow H \rightarrow S$

Better route: $8 < 10$

Known best: 8 This run: 10

A good estimate stays within the truth



What is the true cost left?

$N \rightarrow B \rightarrow H \rightarrow S$

$$\begin{aligned} h^*(N) &= 1 + 2 + 3 \\ &= 6 \end{aligned}$$

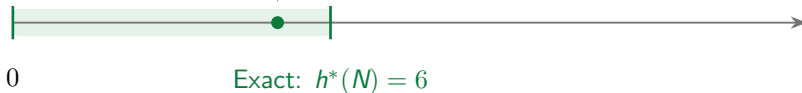
A good estimate stays within the truth



$$0 \leq h(n) \leq h^*(n)$$

Under-estimation

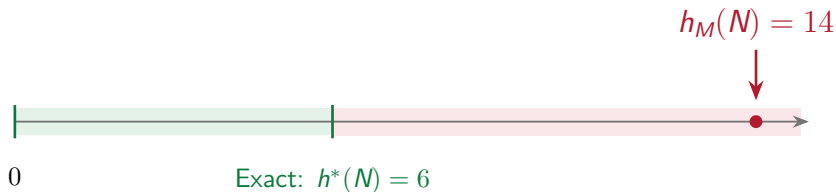
$$h(N) = 5$$



Underestimate or equal the true remaining cost.



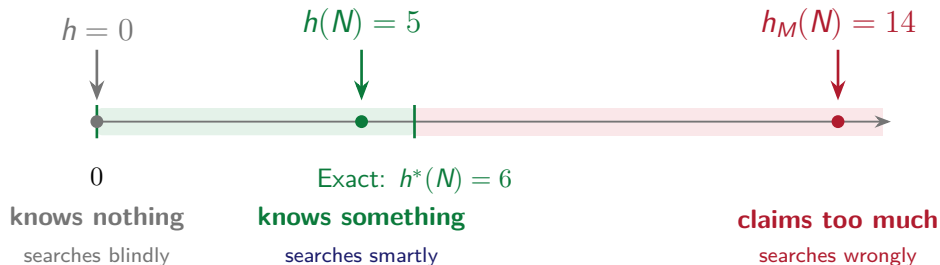
Over-estimation



Optimality is no longer guaranteed.

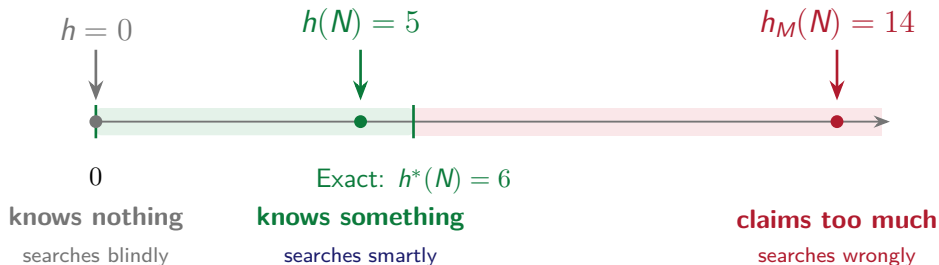
Manhattan overestimates on this weighted road graph.

Not just a shortcut for making A* faster



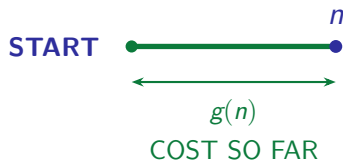


Its quality directly affects **how intelligently** A* searches



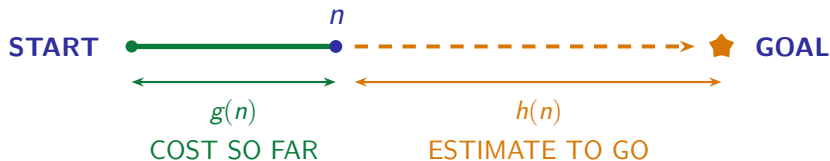


It balances what we KNOW with what we EXPECT



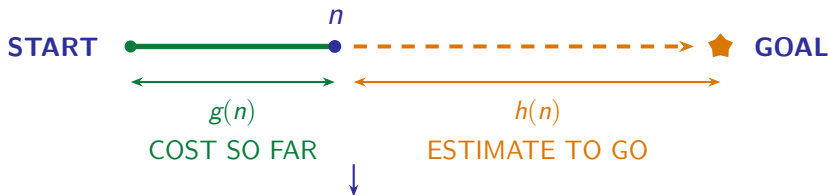


It balances what we KNOW with what we EXPECT





It balances what we **KNOW** with what we **EXPECT**



$$f(n) = g(n) + h(n)$$

A* expands the node with the lowest $f(n)$



Thank You

Every good journey begins with a better next step.

$$f(n) = g(n) + h(n)$$

